

정보시스템설계 **Final Project** 보고서

2021170807 김시현, 2021170814 류근호

목차

- I. 문제 정의
- II. 데이터 선정
- III. 모델 구성
 - 1. Library Import
 - 2. 모델 정확도 향상
 - 3. 최종 모델
- IV. 분석 결과
- V. 결론

I. 문제 정의

꽃 이미지 데이터가 주어졌을 때 Lilly, Lotus, Sunflower, Orchid, Tulip의 5가지의 꽃 중 어떤 꽃에 해당하는지 분류할 수 있는 (모델에 쓰는 방식 ex: regression, CNN 등) 모델을 만드는 것을 목표로 하였다.

II. 데이터 선정

모델을 만들기 위해 kaggle에서 데이터를 선정하였다.

(데이터 출처 : <https://www.kaggle.com/datasets/kausthubkannan/5-flower-types-classification-dataset>)

이 데이터 셋에는 총 5개 종의 꽃 사진의 데이터가 포함되어 있고, 꽃 종류에는 Lilly(백합속), Lotus(연꽃), Orchid(난초), Sunflower(해바라기), Tulip(튤립) 이 있으며 대표 사진은 다음과 같다.

				
Lilly	Lotus	Orchid	Sunflower	Tulip

각각의 꽃이 1000개의 사진 데이터를 가지고 있어 총 5000개의 사진 데이터가 포함되어 있다. 데이터 파일은 'flower_images'라는 폴더 안에 각각의 꽃이 별개의 하위 폴더 5개로 저장되어 있는 형태이며 각 하위 폴더의 이름은 꽃의 이름과 같다. 이 파일을 google drive에 업로드 하여 해당 파일 경로를 다음과 같이 'base_dir'로 저장하였다.

```
base_dir = r'/content/drive/MyDrive/Colab Notebooks/flower_images'
```

III. 모델 구성

1. Library import

도입부에서 이후 필요한 라이브러리들을 모두 import 하였다.

```
import os
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.layers import (
    Activation, Dense, Flatten, Input, Conv2D, MaxPool2D
)
from tensorflow.keras.utils import plot_model
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from IPython.display import Image
from google.colab import drive
drive.mount("/content/drive")
```

2. 모델 정확도 향상

먼저 val_split을 0.3으로 설정하였고 flow_from_directory 함수를 사용하여 각 파일의 이름을 class로 선언한 train_generator와 validation_generator를 생성하였다. 그 결과, 3500장은 train, 1500장은 validation에 사용할 수 있게 세팅되었음을 확인하였다. 이후, Convolutional Neural Network(CNN) 기법을 사용하여 최적의 모델을 찾기 위해 다음과 같은 방법들을 시도하였다.

방법 1 - input size 및 batch 변화시키기

학습 속도 및 예측 성능을 모두 고려한 최적의 target size 및 batch 사이즈를 찾고자 하였다. 이 목표를 달성하기 위해 아래와 같이 parameter의 값을 다르게 설정한 후 각각 학습하는 것을 두 번 반복하였다. model의 구조는 image size 및 batch size에 맞추어 적절히 작성하였으며, epoch는 똑같이 10으로 설정한 후 비교하였다. 설정한 parameter 값과 시행 결과는 다음과 같다.

<모델 1>

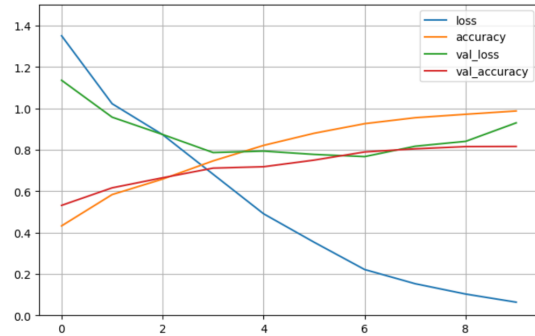
image size=(128,128), batch=64

CNN model summary

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_3 (Conv2D)	(None, 61, 61, 64)	18496
max_pooling2d_3 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_4 (Conv2D)	(None, 28, 28, 128)	73856
max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_1 (Flatten)	(None, 25088)	0
dense_3 (Dense)	(None, 128)	3211392
dense_4 (Dense)	(None, 5)	645

=====
Total params: 3,305,285
Trainable params: 3,305,285
Non-trainable params: 0
=====



loss: 0.0635 - accuracy: 0.9874 - val_loss: 0.9300 - val_accuracy: 0.8160

<모델 2>

앞 모델의 경우 학습 속도가 너무 오래 걸린다는 단점이 있어, 이를 극복하고자 다음과 같이 구상해 보았다.

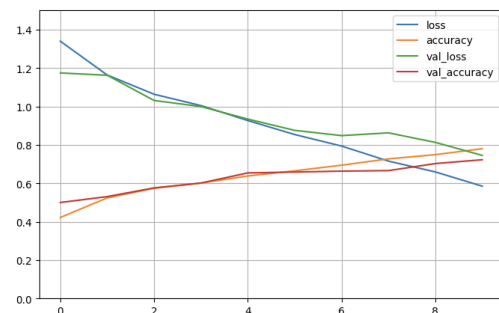
image size=(28,28), batch=32

CNN model summary

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	896
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 128)	0
flatten (Flatten)	(None, 128)	0
dense (Dense)	(None, 128)	16512
dense_1 (Dense)	(None, 5)	645

=====
Total params: 110,405
Trainable params: 110,405
Non-trainable params: 0
=====



loss: 0.5850 - accuracy: 0.7800 - val_loss: 0.7453 - val_accuracy: 0.7227

그러나 **accuracy**를 포함하여 전체적인 성능이 모델1에 비해 현저하게 낮았기 때문에 학습 속도가 느린 것을 감수하더라도 모델2보다는 모델1의 선택이 합리적인 것으로 판단하였다. 결론적으로, 모델1의 image size=(128,128), batch size=64로 설정했던 것이 해당 데이터의 모델 학습에 더 적합하다고 판단할 수 있었기에, 이 설정값을 사용하기로 결정하였다.

방법 2 - 'ImageDataGenerator' 활용

data augmentation 기법을 사용하여 모델을 향상시키기 위해 다음과 같이 설정한 ImageDataGenerator를 통해 데이터 수를 증가시켜 모델을 학습시켜보고자 했다.

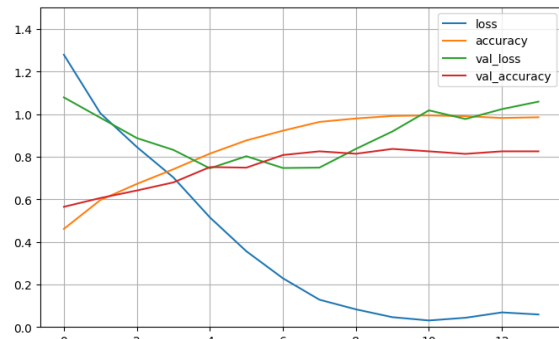
```
train_gen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=20,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    validation_split=val_split  
)
```

Epoch 9/14
55/55 [=====] - 56s 1s/step - loss: 0.8275 - accuracy: 0.6749 - val_loss: 0.8494 - val_accuracy: 0.6727
Epoch 10/14
55/55 [=====] - 61s 1s/step - loss: 0.8158 - accuracy: 0.6831 - val_loss: 0.8475 - val_accuracy: 0.6773
Epoch 11/14
55/55 [=====] - 55s 1s/step - loss: 0.7895 - accuracy: 0.6980 - val_loss: 0.8283 - val_accuracy: 0.6867
Epoch 12/14
55/55 [=====] - 55s 1s/step - loss: 0.7741 - accuracy: 0.6980 - val_loss: 0.7964 - val_accuracy: 0.6960
Epoch 13/14
55/55 [=====] - 55s 1s/step - loss: 0.7331 - accuracy: 0.7109 - val_loss: 0.7982 - val_accuracy: 0.6960
Epoch 14/14
55/55 [=====] - 55s 1s/step - loss: 0.7186 - accuracy: 0.7166 - val_loss: 0.8014 - val_accuracy: 0.7007

그러나 학습 결과는 다음과 같이 기존 모델1의 시행결과보다 더 낮은 성능을 보였고, 따라서 해당 기법은 사용하지 않고 원래대로 진행하였다.

방법 3 - 적정 epoch 탐색

적정 학습횟수를 찾기 위해 epoch을 20으로 설정한 후 모델을 학습시켜 보았고, 11번째 epoch부터 과적합 현상이 나타남을 확인했다. 이후 이 문제를 해결하기 위해 epoch을 14로 지정하고 재학습시켰으나, 그래프에서 보이는 것과 같이 이번엔 6번째 epoch부터 과적합 현상이 일어났다.



이처럼 적정 epoch의 변동이 커 최적값을 찾는데 어려움을 겪었고, 그 대안으로 적절한 시점에 학습을 종료시켜주는 EarlyStopping 함수를 활용하기로 결정하였다. 생성 및 학습 코드는 다음과 같다. epoch은 충분히 큰 크기인 20으로 지정하였다. 그 결과, 적정 epoch은 7로 확인되었다. 그 이유로는 epoch 8 이후부터 accuracy값은 상승했지만 loss 값이 급상승하였기 때문으로 보여진다.

```
early_stopping = EarlyStopping(monitor = 'val_accuracy', min_delta = 0, patience = 1, mode = 'auto')
```

```
history = model.fit(train_generator,  
    epochs=20,  
    validation_data=validation_generator,  
    callbacks = [early_stopping])
```

3. 최종 모델

위의 과정을 거쳐 결정된 최종 입력 값 및 완성된 모델은 다음과 같다.

```
val_split=0.3

train_gen=ImageDataGenerator(rescale=1./255, validation_split=val_split)

train_generator=train_gen.flow_from_directory(
    base_dir,
    target_size=(128,128),
    batch_size=64,
    class_mode='sparse',
    subset='training')

validation_generator=train_gen.flow_from_directory(
    base_dir,
    target_size=(128,128),
    batch_size=64,
    class_mode='sparse',
    subset='validation')
```

Found 3500 images belonging to 5 classes.
Found 1500 images belonging to 5 classes.

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(5, activation='softmax')])
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Layer (type)	Output Shape	Param #
conv2d_34 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_34 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_35 (Conv2D)	(None, 61, 61, 64)	18496
max_pooling2d_35 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_36 (Conv2D)	(None, 28, 28, 128)	73856
max_pooling2d_36 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_12 (Flatten)	(None, 25088)	0
dense_26 (Dense)	(None, 128)	3211392
dense_27 (Dense)	(None, 5)	645

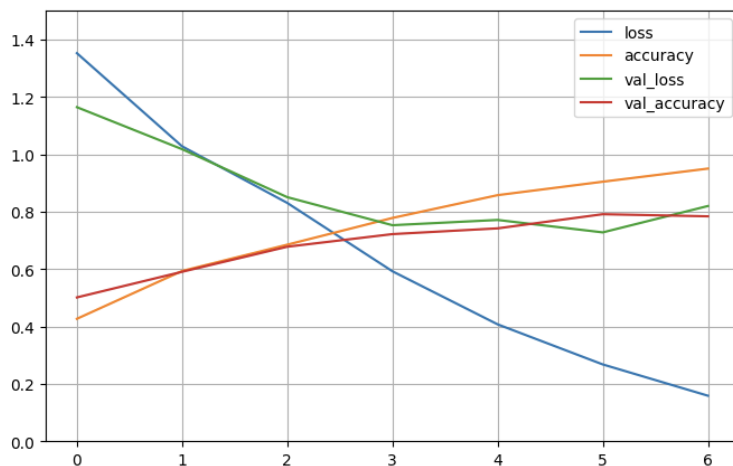
=====

Total params: 3,305,285
Trainable params: 3,305,285
Non-trainable params: 0

```

Epoch 1/20
55/55 [=====] - 40s 702ms/step - loss: 1.3525 - accuracy: 0.4269 - val_loss: 1.1643 - val_accuracy: 0.5013
Epoch 2/20
55/55 [=====] - 38s 694ms/step - loss: 1.0279 - accuracy: 0.5934 - val_loss: 1.0179 - val_accuracy: 0.5907
Epoch 3/20
55/55 [=====] - 38s 686ms/step - loss: 0.8307 - accuracy: 0.6851 - val_loss: 0.8507 - val_accuracy: 0.6780
Epoch 4/20
55/55 [=====] - 38s 694ms/step - loss: 0.5925 - accuracy: 0.7783 - val_loss: 0.7530 - val_accuracy: 0.7220
Epoch 5/20
55/55 [=====] - 39s 709ms/step - loss: 0.4076 - accuracy: 0.8577 - val_loss: 0.7713 - val_accuracy: 0.7420
Epoch 6/20
55/55 [=====] - 38s 687ms/step - loss: 0.2679 - accuracy: 0.9046 - val_loss: 0.7280 - val_accuracy: 0.7913
Epoch 7/20
55/55 [=====] - 38s 689ms/step - loss: 0.1588 - accuracy: 0.9503 - val_loss: 0.8199 - val_accuracy: 0.7840

```



IV. 분석 결과

test를 위해 직접 각 꽃 종류 별 5개의 사진을 선정하여 **test data**를 만들어 사용하였다. 사진과 **class**를 각각 **x_test**, **y_test**에 할당하였고, **labeling**을 위한 딕셔너리를 형성하여 **test_classname**에 저장했다.

```

test_dir = r'/content/drive/MyDrive/Colab Notebooks/test_flower'

testset_generator=train_gen.flow_from_directory(
    test_dir,
    target_size=(28,28),
    class_mode='sparse')

x_test,y_test=testset_generator.next()

testclass_name=dict((a,b) for b,a in train_generator.class_indices.items())
testclass_name

```

아래 사진은 **test data**를 이용해 모델의 성능을 확인한 결과이다.

```

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)

print('\nTest accuracy:', test_acc)

1/1 - 0s - loss: 0.6893 - accuracy: 0.8000 - 27ms/epoch - 27ms/step

Test accuracy: 0.800000011920929

```

test accuracy 값은 약 0.8로 비교적 높은 정확도를 보여주는 것을 확인할 수 있다.

test 및 결과 확인을 위해 각 test image의 예측 성공 여부 및 prediction의 probability를 모두 표시하여 출력해 보고자 했다. 이를 위해 두 종류의 함수를 작성하였다. plot_image 는 입력된 사진과 '예측값, prediction probability, (정답)'의 형태의 문구가 출력되는 함수이고, plot_value_array는 예측분포를 그래프로 나타내는 함수이다.

```
probability_model = tf.keras.Sequential([model, tf.keras.layers.Softmax()])
predictions = probability_model.predict(x_test)
```

```
def plot_image(i, predictions_array, true_label, img):
    true_label, img = true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

    plt.imshow(img, cmap=plt.cm.binary)

    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'

    plt.xlabel("{} {:2.0f}% ({})".format(testclass_name[predicted_label],
                                        100*np.max(predictions_array),
                                        testclass_name[true_label]),
               color=color)

def plot_value_array(i, predictions_array, true_label):
    true_label = true_label[i]
    true_label=int(true_label)
    plt.grid(False)
    plt.xticks(range(5))
    plt.yticks([])
    thisplot = plt.bar(range(5), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

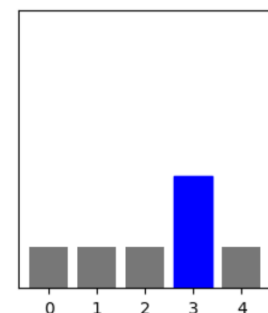
    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')
```

첫 번째 데이터만 대표로 입력하여 확인해 본 결과이다. 예측과 실제 꽃 이름이 일치하는 경우, 그래프 및 글씨가 파란색으로 표시되며 오답인 경우 예측된 값(오답)은 빨간색, 정답은 파란색으로 나타난다. 이 경우 sunflower 사진을 40%의 probability로 알맞게 예측한 것을 확인할 수 있다.

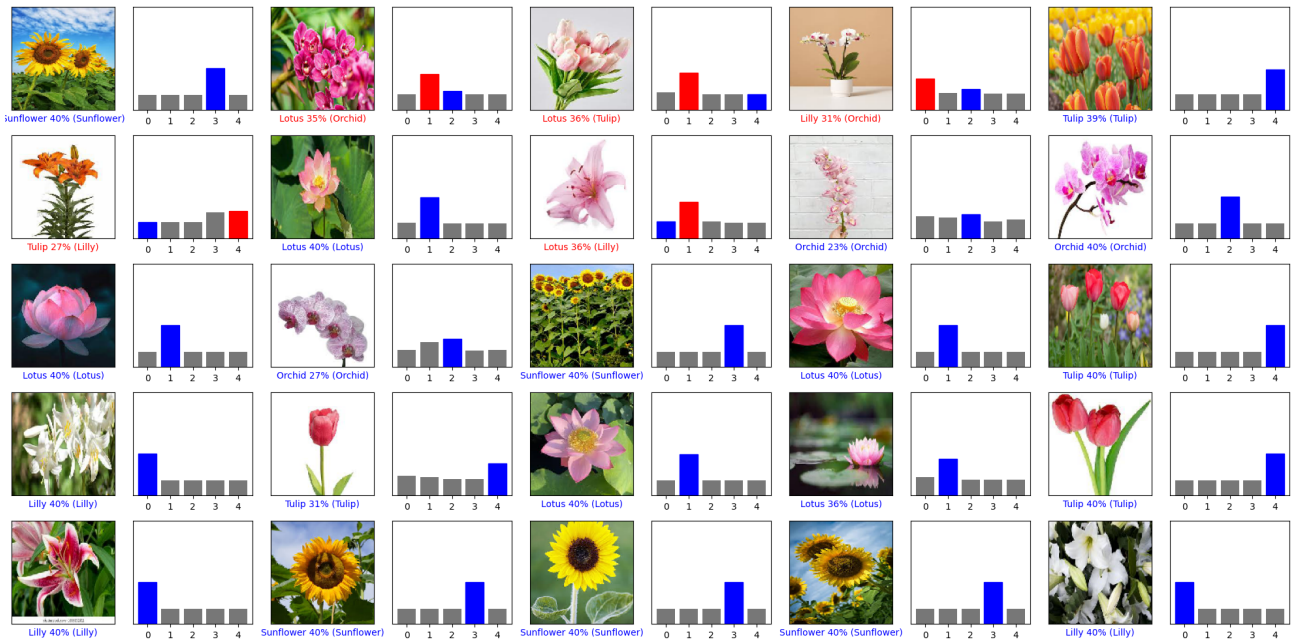
```
i = 0
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plot_image(i, predictions[i], y_test, x_test)
plt.subplot(1,2,2)
plot_value_array(i, predictions[i], y_test)
plt.show()
```



Sunflower 40% (Sunflower)



아래 사진은 25개 데이터를 모두 test하여 하나의 plot으로 나타내 본 결과이다.



V. 결론

1. 결과

최종 실험의 결과로 accuracy는 0.9503, validation accuracy는 0.7840으로 validation dataset에 대해 상대적으로 낮은 정확도이긴 하나, 전체적으로는 높은 정확도를 가진 모델을 얻어낼 수 있었다. 이 모델로 25개(꽃 종류별 5개)의 dataset을 가지고 test를 진행한 결과, 0.8의 accuracy라는 높은 정확도로 5개의 class로 분류해낼 수 있다는 것을 확인했다.

2. 느낀 점 및 한계

모델이 전체 3,305,285개의 parameter를 가지는 것에 반해 5000개라는 학습 data의 수는 상대적으로 부족했다. 그러나, 5000개의 dataset을 학습시키는데에도 상당한 양의 시간이 소요돼 처음 만들었던 모델을 간소화 해야할 필요성을 느끼기도 했다는 점에서 주어진 환경에서는 최선을 다했다고 생각한다. 다만, 이로 인해 test에서 더 높은 accuracy를 얻어내지 못했다는 점에서 아쉬움을 느끼는 바이다.

다만, 최적 epoch값을 찾는 과정에서 overfitting 문제가 발생하는 것을 경험하며 이를 해결하는 과정에서 많은 것을 느낄 수 있었다. 무엇보다 모델의 accuracy와 별개로 해당 문제가 발생할 경우 예측력이 현저히 떨어진다는 것을 직접 확인해 보며 overfitting의 개념 및 영향에 대한 이해도를 상승시킬 수 있었고, 더불어 deep learning 모델에 대한 전체적인 이해도가 상승하고 다양한 부가 함수들을 경험할 수 있었다.

마지막으로 주어진 기간내에 한정된 GPU 자원을 가지고 적절한 **layer**를 찾아야 하다보니 조금 더 좋은 성능의 모델을 만들 수도 있을 것 같다는 아쉬움이 남는다. 다만 **layer** 구성을 조금씩 바꿔가며 모델에 따라 크게 변화하는 **accuracy, loss**, 그리고 학습에 소요되는 시간 등을 확인 및 비교하였고, 그 중 가장 적절한 조합을 찾기 위해 노력하였다는 것에 의의가 있다.